

Lab Guide — Lakeflow Connect & Orchestration

Work top to bottom. Everything lives in **your** schema inside the `dev-sh-training` catalog. Set the `schema` widget to your own name on every notebook before you run it.

- Catalog (fixed): `dev-sh-training`
- Your schema: whatever you set in `00_setup` (e.g. `jane_doe`)
- Landing volume: `/Volumes/dev-sh-training/<you>/landing/`

⚠ How to use every notebook in this lab — read first. The widgets (`schema`, `scenario`, ...) are defined in the **first cell**. You must **run that first cell** before anything else so the widget boxes appear at the top of the notebook. Then fill in your `schema` (and any other options) and run the remaining cells (or **Run all**). If you skip the first cell, the widgets never appear, and the notebook falls back to the placeholder default and stops with an error.

The value you type is substituted everywhere automatically — via `${schema}` in SQL notebooks and via the widget in Python notebooks.

Setup (once)

1. Add this repo as a **Git folder** (Workspace → Create → Git folder, URL `https://github.com/realboom/lakeflowconnect`).
2. Open `notebooks/00_setup`, **run the first cell** so the `schema` widget appears, set it to your name, then **Run all**. You now have a `schema` and a `landing` volume with `filedrop/` and `sftp/` subfolders.

Part A — COPY INTO (batch file ingestion)

Concept: `COPY INTO` loads *new* files from a folder into a table and is **idempotent** — rerun it and already-loaded files are skipped. Simple, batch, great for “a file shows up, pull it in.”

1. Open `generate_data`, set `schema`, `scenario` **Part A - file drop (COPY INTO)**, `num_rows` = 25, **Run all**. → writes a comma CSV to `landing/filedrop/incoming/`.
2. Open `part_a_copy_into`, set `schema`, **Run all**. → creates `bronze_file_eligibility` and loads the file.
3. **Run the COPY INTO cell again.** It loads **0 new rows** — that’s idempotency (it remembers what it already ingested).

4. Generate another Part A file (step 1 again) and re-run COPY INTO → only the **new** file is loaded.

✅ Takeaway: COPY INTO = the simplest incremental batch ingest.

Part B — Auto Loader (“SFTP” source, mimicked on a Volume)

Concept: A vendor normally drops files on **SFTP**. We mimic that by landing pipe-delimited files in `landing/sftp/incoming/` and let **Auto Loader** (`cloudFiles`) ingest them incrementally. Auto Loader keeps a **checkpoint** of what it has processed and is built for continuously-arriving files.

1. Open `generate_data`, scenario **Part B – SFTP source (Auto Loader)**, `num_rows` = 25, **Run all.** → writes a **pipe-delimited** CSV to `landing/sftp/incoming/`.
2. Open `part_b_autoloader`, set `schema`, **Run all.** → creates `bronze_sftp_eligibility` and ingests the file. The last cells show the data.
3. Generate another Part B file and re-run Part B → only the **new** file is picked up (checkpoint).

✅ Takeaway: Auto Loader = incremental, checkpointed ingestion — the standard for flat-file / “SFTP” feeds. Contrast with Part A: COPY INTO is batch SQL; Auto Loader is a streaming source with schema evolution + rescue (next).

Part C — Auto Loader resilience

Same `bronze_sftp_eligibility` pipeline from Part B. We show the two ways Auto Loader absorbs drift. **Do rescued data first** (it runs on the original schema), then schema evolution (it permanently adds a column).

C1 — Rescued data (bad value, original schema)

1. `generate_data` → scenario **Part C – rescued data (SFTP)** → **Run all.** Half the rows have a bad `eff_date` of `N0T-A-DATE`. (`eff_date` is typed `DATE` via `schemaHints`.)
2. `part_b_autoloader` → **Run all.** The bad values can’t cast to `DATE`, so Auto Loader **rescues** them: the row still ingests with `eff_date = NULL`, and the original value is parked in `_rescued_data`.
3. The last cell shows the rescued rows.

💡 A malformed value didn’t crash the pipeline or get silently dropped — it’s captured for audit/backfill.

C2 — Schema evolution (a new column)

1. `generate_data` → scenario **Part C - schema evolution (SFTP)** → **Run all**. The file has a new `risk_tier` column.
2. `part_b_autoloader` → **Run all**. Auto Loader detects `risk_tier`. In a notebook it **stops once** to record the new column — if you see an “unknown field”/schema-change message, **just run the cell again** and it succeeds, with `risk_tier` added to the table (existing rows `NULL`).
3. `SELECT risk_tier, count(*) ... GROUP BY risk_tier` to confirm.

🔔 A new field appeared and the table evolved on its own. (In Part E a task **retry** makes this one-run automatic.)

Part E — Orchestration (Lakeflow Pipelines + Jobs)

Now we orchestrate the eligibility data you just created. You’ll build **one pipeline** and a few **jobs** by hand.

Where you build these — read this once. In the Databricks **left sidebar**, find **Jobs & Pipelines**. Tip: **right-click it and open in a new browser tab**, so you can keep this guide open side-by-side. On the **Jobs & Pipelines** page, click the blue **Create** button (it has a dropdown). The dropdown lets you pick what to build — **ETL Pipeline** or **Job**. Each step below tells you which one to choose: - **E1** builds an **ETL Pipeline** (that’s what a Spark Declarative Pipeline is). - **E2, E3, E5** build **Jobs**.

Picking a notebook for a task — read this once too. Whenever a task asks for a notebook **Path**, the **Select Notebook** dialog opens. Navigate **Workspace** → **Users** → **your user** → `lakeflowconnect` → `notebooks`, select the file, and click **Confirm**. (`lakeflowconnect` is the Git folder you cloned in Setup.) Below, a reference like `notebooks/gold_eligibility_summary` means that file in this folder.

E1 — The silver pipeline (an ETL Pipeline / Spark Declarative Pipeline)

This builds a streaming `silver_eligibility` table from your Auto Loader bronze (`bronze_sftp_eligibility`).

1. **Jobs & Pipelines** (left sidebar) → blue **Create** button → **ETL Pipeline**. You land in the pipeline editor on a new pipeline named like *New Pipeline 2026-...*, with a starter file `transformations/my_transformation.py`.
2. **Rename the pipeline:** click its name at the top-left and change it to `silver_eligibility_<you>`. If it asks “**Also rename the root folder?**”, click **Rename** (keeps the pipeline’s workspace folder name in sync — either choice works).
3. ⚠️ **Set the default catalog + schema — this is how the pipeline finds your tables.** Open the pipeline **Settings** (gear icon, or the `:` menu → *Settings*) and set:
 - **Default catalog** = `dev-sh-training`

- **Default schema = your schema** (the one from `00_setup`)
- **Serverless** compute

(The editor shows a default like `... / default` near the top-right — you must change it to yours.) Because the code uses the bare name `bronze_sftp_eligibility` (no catalog/schema), it resolves against this default catalog + schema — so your pipeline reads *your* bronze and writes *your* silver. Same code for everyone; only the default schema differs.

4. **Add the transformation SQL.** The starter file is Python, but our logic is SQL:

- Right-click `my_transformation.py` → **Rename** to `silver_eligibility.sql` (or click + to add a new `silver_eligibility.sql` and delete the empty `.py`).
- Open `notebooks/silver_eligibility` in your Git folder, copy the `CREATE OR REFRESH STREAMING TABLE silver_eligibility ...` statement, and paste it into the file.
- (Optional — AI authoring: instead of pasting, click **Create with Genie Code / Generate** and paste this prompt — then compare what it writes to the SQL in the repo before running: > “Create a streaming table in SQL named `silver_eligibility` that reads incrementally from the > streaming table `bronze_sftp_eligibility`. `initcap` `first_name` and `last_name`; cast `eff_date` and > `term_date` to DATE; keep `member_id`, `plan_code`, `line_of_business`; add `silver_loaded_at` as > the current timestamp. Use `FROM STREAM bronze_sftp_eligibility` so it’s incremental, not a reload.”)

5. Click **Run** (▶ Run pipeline) → it builds `silver_eligibility` in your schema from `bronze_sftp_eligibility`. (Part B must have run first so your bronze table exists.)

E2 — Table-update trigger (event-driven)

Make the silver pipeline run **automatically whenever new bronze data lands** — no schedule, no manual run.

1. **Jobs & Pipelines** (left sidebar) → blue **Create** button → **Job**. You land in the job editor on a job named like *New Job 2026-....*
2. **Rename the job:** click its name at the top-left → `silver_refresh_<you>`.
3. **Add the task — and make it a Pipeline task** (the screen defaults to a Notebook card; don’t use that):
 - Click + **Add another task type** (the blue button under “Add your first task”).
 - **Task name:** `refresh_silver`
 - **Type:** choose **Pipeline** from the Type dropdown.
 - **Pipeline:** select your `silver_eligibility_<you>` pipeline from E1.
 - **Create task / Save task.**
4. **Add the trigger** — it lives in the **right-hand panel**, not on the task:
 - In the right-side **Job details** panel, find **Schedules & Triggers** and click **Add trigger**.
 - **Trigger type:** **Table update**.
 - **Tables:** click the table box and browse to `dev-sh-training` → **your schema** → `bronze_sftp_eligibility`. ⚠ Make sure it’s `bronze_sftp_eligibility` (the Auto Loader table your silver pipeline streams from) — **not** `bronze_file_eligibility` (that’s

Part A's COPY INTO table; silver doesn't read it, so the job would fire but silver would see nothing new).

- **Advanced** is optional. The **Minimum time between triggers** / **Wait after last change** fields are in **hh mm** (not seconds) — leave both at **00h 00m** so the job fires as soon as new data lands.
- **Save**.

5. Test it:

- Click the **Runs** tab (next to **Tasks**, just under the job name) so you can watch runs appear.
- In another browser tab, run `generate_data` (scenario **Part B - SFTP source**) then `part_b_autoloader` so new rows commit to `bronze_sftp_eligibility`. (Plain Part B is all you need — you just want a fresh commit to fire the trigger. `risk_tier` already exists from Part C and silver doesn't use it, so there's no need to re-run schema evolution here.)
- Within ~a minute a run **shows up on the Runs tab on its own** — its origin reads *"Triggered by table update"* — and it refreshes `silver_eligibility`. (No one pressed Run now.)

💡 The trigger baselines when you create it and fires only on **new** commits — so do the fresh bronze load *after* you've set up the trigger, or it won't have anything new to react to.

E3 — The DAG: gold → report → notify (with a conditional branch)

Create a multi-task job. 1. **Jobs & Pipelines** → blue **Create** button → **Job**, name it `eligibility_gold_<you>` (include your name — jobs are workspace-global, so a bare `eligibility_gold` would collide with everyone else's). 2. Task **gold** — Type **Notebook** → `notebooks/gold_eligibility_summary`, param `schema` = your schema. Under **Advanced** → **Retries**, set **Max retries** = 2. 3. Task **report** — Type **Notebook** → `notebooks/eligibility_report`, param `schema` = your schema. **Depends on** `gold`. 4. Task **notify_pass** — Notebook `notebooks/notify`, param `result` = `pass`. Depends on `report`. **Run if** = **All succeeded**. 5. Task **notify_fail** — Notebook `notebooks/notify`, param `result` = `fail`. Depends on `report`. **Run if** = **At least one failed**. 6. **Run** the job → `gold` → `report` → `notify_pass` (and `notify_fail` is skipped). Green path.

E4 — Repair / partial re-run (recover from a schema change)

A realistic failure: someone renames a column upstream and the downstream report breaks. 1. Open `gold_eligibility_summary` and rename the output column `member_count` → `enrolled_members`, save. 2. **Run** `eligibility_gold_<you>`. → `gold` **succeeds** (table rebuilt with the new name), but `report` **fails**: `UNRESOLVED_COLUMN ... member_count ... Did you mean 'enrolled_members'?`, and `notify_fail` runs. 3. **Fix** `eligibility_report`: change `member_count` → `enrolled_members`, save. 4. On the failed run click **Repair run**. → `gold` **is skipped** (already built), only `report` + `notify_pass` re-run → success.

🔔 You don't rerun the expensive upstream that already succeeded — fix and resume from the failure. (To reset for a repeat: revert both files back to `member_count`.)

E5 — For each (fan-out + concurrency)

Run a task once per line of business, in parallel. 1. **Jobs & Pipelines** → blue **Create** button → **Job eligibility_by_lob_<you>**. Add a **For each** task. - **Inputs:** `["Commercial","Medicaid","Medicare Advantage","Individual"]` - **Concurrency:** 4 2. **Add a task to loop over** → Notebook `notebooks/for_each_lob`, params `schema` = your schema and `lob = {{input}}`. 3. **Run** → four iterations run in parallel, each building `gold_elig_<lob>`.

Concurrency FAQ (you'll be asked): concurrency is **1–100** (default 1); effective parallelism = `min(concurrency, # inputs)`; the workspace cap is **2,000 concurrent task runs**. Size it to what your downstream can handle, not the max. See the instructor runbook for the full FAQ.

E6 — Retries

Notice the **Max retries = 2** you set on the `gold` task. Retries heal transient failures — and they're exactly what makes the Part C **schema-evolution** handshake complete in a single automated run (Auto Loader stops once to record the new column; the retry picks it up). Put a retry on any task that ingests evolving files.

E7 — Incremental loading: upsert with AUTO CDC (*optional — a preview of the SDP workshop*)

The E1 `silver_eligibility` table is **append-only**: re-send a member and you get a **duplicate** row. A “current state” table (eligibility, customer, account) usually wants **one row per member, updated in place**. Databricks does this with **AUTO CDC** (the SQL `APPLY CHANGES` pattern) — you declare the **key** and a **sequence**, and Databricks writes the insert-or-update MERGE for you (no hand-coded MERGE, no full reload).

1. Build a **second ETL Pipeline**, set up exactly like E1 (Create → ETL Pipeline → rename, e.g. `silver_upsert_<you>` → **default catalog dev-sh-training + your schema** → Serverless), but paste `notebooks/silver_eligibility_scd` as the source. It creates `silver_eligibility_current` (one upserted row per `member_id`). Click **Run**.
2. Note a few members and their plans: `SELECT member_id, plan_code FROM silver_eligibility_current ORDER BY member_id LIMIT 5;`
3. **Send updates:** `generate_data` → scenario **Part E – member update (upsert)** → Run all. It re-sends existing members with a **new plan_code**. Then run `part_b_autoloader` so the changes land in bronze.
4. **Run the AUTO CDC pipeline again** (or wire a table-update trigger like E2 to fire it). Re-check the same members — their `plan_code` has **changed in place**: same row count, **no duplicates**.

5. **See the contrast:** the append `silver_eligibility` from E1, fed the same update, shows **two rows** for that member (old + new). Append vs. upsert is the core incremental-loading choice.

The three pieces to call out: `KEYS (member_id)` (match on the business key), `SEQUENCE BY ingested_at` (newest wins when a key repeats), `AUTO CDC INTO` (Databricks runs the MERGE). This is the foundation we go deep on in the SDP workshop — upserts, deletes (`APPLY AS DELETE`), and SCD Type 2 history.

Clean up / start over

Run `notebooks/reset` (set your `schema`) to drop the lab tables and clear the landing folders. Your schema and volume stay. Re-run `generate_data` to begin again.